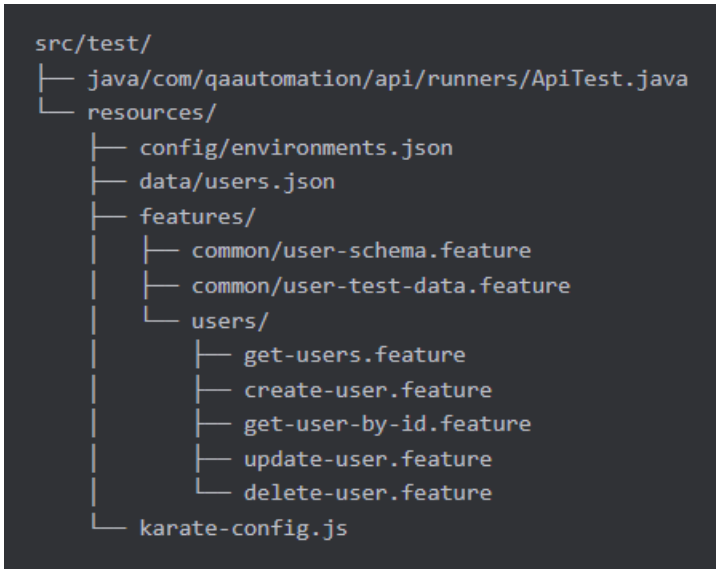
Reto de Automatización QA - BackEnd

1. Objetivo

La automatización tiene como objetivo validar la API de usuarios de ServeRest utilizando Karate DSL, cubriendo las operaciones CRUD. Se contemplan escenarios positivos y negativos, validación de códigos HTTP, mensajes y estructura de las respuestas JSON.

2. Estrategia

Los features están organizados por endpoint para facilitar su lectura y mantenimiento:



Se mantiene una separación de responsabilidades entre configuración de ambientes, datos de prueba, schemas JSON y escenarios funcionales.

3. Datos de prueba

`user-test-data.feature` funciona como un helper para centralizar los payloads utilizados por los escenarios.

Se generan datos dinámicos mediante:

```
* def timestamp = java.lang.System.currentTimeMillis()
```

Esto permite generar valores únicos, como los emails utilizados durante las pruebas, reduciendo conflictos entre ejecuciones.

4. Validación de respuestas

Los schemas reutilizables se encuentran en `features/common/user-schema.feature`. Se utilizan para validar la estructura y los tipos de datos de las respuestas:

```
And match response == schemas.userSchema
```

También se validan respuestas de colecciones, mensajes y escenarios negativos.

5. Patrones y principios utilizados

Se utiliza un enfoque Factory para datos de prueba, mediante `user-test-data.feature`, permitiendo centralizar y reutilizar los payloads.

También se aplica separación de responsabilidades, manteniendo independientes la configuración, los datos, los schemas y los features funcionales.

No se utiliza Page Object Model, debido a que es un patrón orientado principalmente a automatización de interfaces de usuario y este proyecto está enfocado en una API REST.

6. Ejecución

La suite completa puede ejecutarse mediante:

```
mvn clean test
```

Los reportes HTML generados por Karate se encuentran en:

```
target/karate-reports/
```